



1506
UNIVERSITÀ
DEGLI STUDI
DI URBINO
CARLO BO

UNIVERSITÀ DEGLI STUDI DI URBINO CARLO BO

Dipartimento di Scienze Pure e Applicate
Corso di Laurea Magistrale in Informatica e Innovazione Digitale

Tesi di Laurea

CREAZIONE DI UN GESTORE DI MESSAGGI PER LA SOMMINISTRAZIONE DI TEST IN ERLANG

Relatore:
Chiar.mo Prof. Claudio Antares Mezzina

Candidato:
Paride Dominici

Correlatore:
Chiar.mo Prof. Alessandro Aldini

Anno Accademico 2024-2025

Alla mia famiglia

Indice

1	Introduzione	1
1.1	Scopo del progetto	1
1.2	Introduzione al problema	1
1.3	Possibili soluzioni	2
1.3.1	Aumento delle risorse	2
1.3.2	Allocazione dinamica delle risorse	2
1.3.3	Un nuovo approccio	3
1.3.4	Configurazione	4
2	Analisi dei requisiti	5
2.1	Comunicazione	5
2.1.1	Esempi di messaggi che possono essere utilizzati	5
2.1.2	Requisiti tecnici	6
2.2	Differenze tra le due modalità	6
2.3	Diagrammi degli stati	7
2.4	Perché scegliere Erlang	9
2.4.1	Erlang/OTP	9
2.4.2	Invio e ricezione di messaggi	10
2.4.3	Hot Code Swapping	13
2.4.4	Casi d'uso - Whatsapp	13
2.4.5	Esempio codice del framework OTP	13
3	Strutture di comunicazione	16
3.1	Struttura JSON	16
3.2	Struttura Record in Erlang	17
4	Implementazione della comunicazione	18
4.1	Comunicazione tra client e gestore dei messaggi	18
4.1.1	Lato client Javascript	18
4.2	Lato gestore dei messaggi	20
4.2.1	Gestione delle diverse tipologie di connessione	21
4.2.2	Memorizzazione dei messaggi	22
4.3	Invio dei messaggi da parte del gestore dei messaggi	25

4.3.1	Comunicazione sincrona	25
4.3.2	Comunicazione asincrona	28
4.4	Comunicazione tra gestore dei messaggi e server PHP	29
4.4.1	Limitazione del numero di connessioni al server PHP	29
4.4.2	Ricezione e gestione dei messaggi nel server PHP	31
5	Verifica della funzionalità	33
5.1	Verifica connessione	33
5.1.1	Verifica connessione di base	33
5.1.2	Verifica connessione con gestore messaggi	34
5.2	Stress test	34
5.2.1	Creazione dei file per i test	34
5.2.2	Esecuzione del test	37
6	Conclusioni	39
A	Appendice A	40
A.1	Client javascript per Verifica connessione di base	40
A.2	Client javascript per Verifica comunicazioni tra client e gestore dei messaggi	42
	Bibliografia	43
	Ringraziamenti	44

Elenco delle figure

1.1	Configurazione standard	2
1.2	Nuova configurazione	3
2.1	Esempio comunicazione senza gestore messaggi	6
2.2	Esempio comunicazione con gestore messaggi	7
2.3	Diagramma degli stati del Client	8
2.4	Supervision Tree	10
2.5	Comunicazione ad attori	11
5.1	Test comunicazione di base in modalità GET	33
5.2	Test comunicazione di base in modalità POST	34
5.3	Risultati dello stress test	38

Elenco delle tabelle

3.1	Elenco dei comandi che possono essere inviati dal client	16
3.2	Elenco dei comandi di risposta da parte dei server	17
4.1	Struttura dei record della tabella mnesia	23

Capitolo 1

Introduzione

1.1 Scopo del progetto

Il progetto si propone l'obiettivo di verificare la possibilità di utilizzare le peculiarità del linguaggio Erlang [3] o Elixir [4] per migliorare la gestione dei trasferimenti dei dati nella somministrazione di test.

Nello specifico se sia possibile migliorare e rendere più robusto ed affidabile il trasferimento dei dati dalle postazioni degli studenti al server che si occupa di raccogliere ed elaborare i risultati.

I problemi principali che ci proponiamo di risolvere sono l'affidabilità nel trasferimento dei dati a causa di problemi di connettività e la garanzia di congruenza dei dati stessi.

Per fare questo si vuole creare una rete di nodi (da stabilire) che permettano alle postazioni studente di inviare sia le versioni temporanee che quelle definitive delle risposte al test, ai server che raccolgono i risultati e realizzare anche, sempre tramite la rete di nodi erlang una rete affidabile e resistente ai problemi.

1.2 Introduzione al problema

Al momento la somministrazione dei test al PC utilizzando i browser web utilizzano una architettura di tipo Client/Server per gestire la comunicazione tra il server che gestisce e distribuisce il test e i vari client che registrano ed inviano le risposte inserite dagli studenti.

In linea di massima lo schema è quello visibile in figura 1.1.

In questa modalità ogni client fa una serie di chiamate al server web che genera dinamicamente la pagina da visualizzare sul browser del client. Questa pagina viene generata diversamente in base allo stato in cui si trova il client. Ad esempio potrebbe essere la pagina per fare il login alla piattaforma oppure quella in cui si inseriscono i propri dati passando per le pagine in cui ci sono le domande del test con i campi in cui l'utente inserisce la risposta esatta fino ad arrivare alla pagina con il punteggio realizzato del test.

Tutte queste interazioni avvengono, attraverso chiamate http o https e sfruttano direttamente le risorse della macchina su cui è installato il web server.

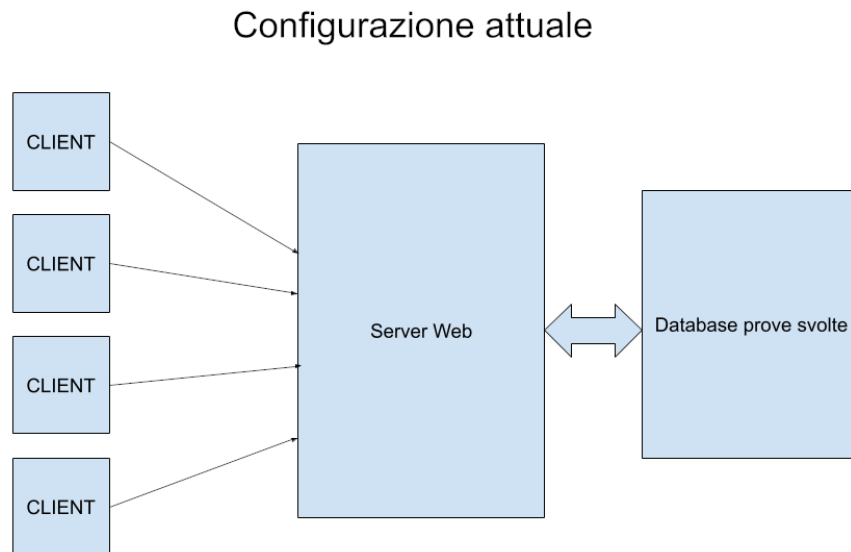


Figura 1.1: Configurazione standard

All'aumentare del numero di utenti che si collegano aumenta anche la quantità di risorse che vengono utilizzate fino ad arrivare ad un punto, dipendente dall'architettura e dalle risorse presenti sulla macchina, che impedisce al web server di funzionare correttamente provocando una serie di errori.

1.3 Possibili soluzioni

1.3.1 Aumento delle risorse

La prima soluzione e quella più comune è quella di aumentare le risorse disponibili per il Web server.

Questo si può fare acquistando nuovo hardware e installando sulla nuova macchina un nuovo web server che però va configurato ed ottimizzato correttamente e richiede un investimento periodico, oltre che economico per l'acquisto dell'hardware, anche di tempo per l'installazione, la configurazione e il trasferimento dei dati tra una macchina e l'altra.

1.3.2 Allocazione dinamica delle risorse

Un'altra soluzione può esser quella di installare il nostro web server, fin dal primo momento, in un sistema che permetta una scalabilità delle risorse dinamica senza essere legati direttamente all'hardware presente.

Un primo livello è creare una *server farm* interna all'azienda che permetta la gestione delle macchine in ambiente virtuale. In questo caso lo sfruttamento delle risorse viene gestito e diviso, tra le risorse disponibili, da una infrastruttura di base che nasconde il vero hardware.

Un secondo livello è quello di affidarsi a sistemi distribuiti a livello geografico come Amazon AWS, Microsoft Azure, ecc. che permettono anche di allocare dinamicamente le risorse in base al carico di lavoro che è in quel momento presente sul nostro sistema.

In questi casi l'investimento dal punto di vista temporale, rispetto al caso precedente, viene quasi annullato dato che è sufficiente, per aumentare le risorse cambiare qualche parametro anche a macchine accese, per assegnare le risorse necessarie a sostenere il carico di lavoro che si presenta.

D'altro canto l'investimento economico in questi ultimi due casi è necessariamente più grande di quello richiesto dal primo caso di cui abbiamo parlato.

1.3.3 Un nuovo approccio

Il lavoro che verrà portato avanti in questa tesi si pone come obiettivo quello di trovare un sistema per evitare il problema del sovraccarico del server web da parte di troppe connessioni contemporanee senza dover investire in nuove risorse hardware.

Per far questo creeremo in layer intermedio tra il web server e i client come in Figura 1.2.

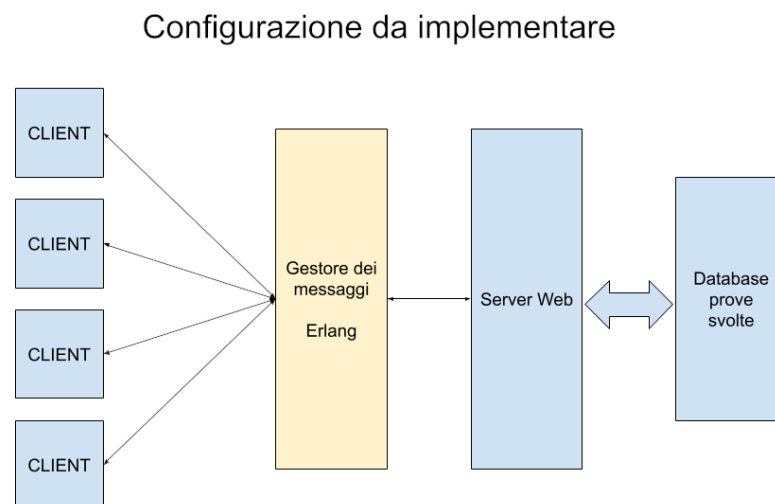


Figura 1.2: Nuova configurazione

Questo nuovo layer deve occuparsi, usando dei messaggi, di fare da tramite tra i client e il server web gestendo le richieste in modo che non ne arrivino troppe contemporaneamente al web server.

Sfortunatamente questo richiederà anche di cambiare il modo in cui i client e il server comunicano tra di loro.

In particolare non sarà più direttamente il server a generare la pagina da visualizzare sul client ma passeremo ad una gestione più dinamica della creazione delle pagine direttamente sul client in Javascript con un approccio di tipo Client Side Rendering o meglio ancora DOM Manipulation.

Questo approccio richiede che il client abbia un browser più moderno dato che una parte del processo di elaborazione viene spostata sul client. D'altro canto ormai tantissime applicazioni, come ad esempio Google mail o Facebook, hanno questo tipo di approccio.

Gli scambi tra il client, il gestore di messaggi e il server PHP, avverranno quindi solo tramite messaggi codificati in JSON e non come pagine web intere. Questo ci permetterà, in ottica futura, di poter modificare una singola parte (client, server o gestore di messaggi) senza dover modificare obbligatoriamente le altre.

1.3.4 Configurazione

Per verificare il nostro progetto realizzeremo un server di test nel quale installeremo il server Yaws nel quale girerà il codice Erlang ed anche una configurazione standard di Apache [9], PHP [10], MySQL [7] o MariaDB [8] in modo da poter provare l'efficacia del nostro sistema.

Implementeremo comunque, sia lato client, che lato server PHP, un set minimo di comandi sufficiente per provare la risposta del server PHP interrogato direttamente dal client html confrontato con la stessa configurazione ma con il gestore di messaggi inserito tra i due elementi.

Capitolo 2

Analisi dei requisiti

2.1 Comunicazione

La prima cosa che dobbiamo analizzare è la tipologia di messaggi che possono essere scambiati tra il gestore dei messaggi, i client e il server che gestisce i test.

Per avere un riferimento stiliamo innanzitutto un elenco dei messaggi che potrebbero essere scambiati tra il server e i client in una situazione standard.

2.1.1 Esempi di messaggi che possono essere utilizzati

- Registrazione nuova postazione
- De-registrazione postazione
- Richiesta codice di accesso
- Invio codice di accesso
- Richiesta invio dati studente
- Invio dati studente
- Richiesta test da somministrare
- Invio contenuto del test da somministrare
- Invio risposte test
- Termine forzato del test
- Mostra messaggio sulla postazione
- Richiesta stato postazione

2.1.2 Requisiti tecnici

Per poter comunicare sia lato client che lato server, visto che le comunicazioni avvengono già su http o https non useremo erlang direttamente ma ci affideremo al web server Yaws [5] che è stato implementato proprio in Erlang e che permette di inserire all'interno del codice html, in modo simile a PHP, direttamente chiamate a codice Erlang. Saremo costretti ad adattare il codice del client e del server in modo che comunichino attraverso il nuovo gestore dei messaggi.

2.2 Differenze tra le due modalità

La differenza principale, nella comunicazione tra client e server è che nella configurazione originale, senza il gestore dei messaggi, ogni richiesta del client viene gestita direttamente dal server che risponde, nella stessa connessione, con la pagina o il messaggio relativo.

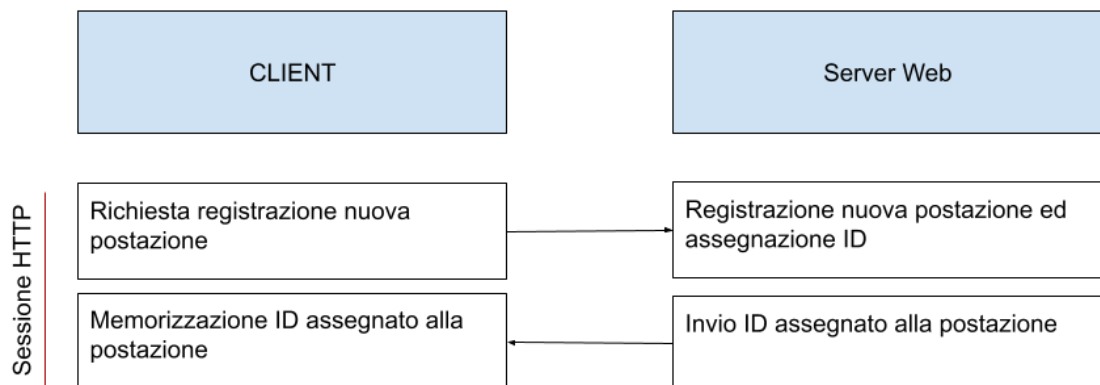


Figura 2.1: Esempio comunicazione senza gestore messaggi

Nella configurazione con il gestore dei messaggi invece la comunicazione avviene in modo asincrono. Il client fa una richiesta al gestore dei messaggi, questo risponde nella stessa comunicazione con l'ID con il quale la richiesta è stata memorizzata nella coda dei messaggi in modo che il client possa sapere quando la richiesta è stata processata.

Ad intervalli regolari il client farà una richiesta al gestore dei messaggi per conoscere l'esito della sua richiesta.

La comunicazione tra gestore dei messaggi e server PHP avverrà in modo simile. Il gestore dei messaggi prenderà uno dei messaggi dalla coda e si collegherà al server PHP. Per evitare le congestioni implementeremo un sistema che limiti il numero di processi contemporanei attivi. Se tutte le connessioni che abbiamo deciso fossero già occupate il gestore di messaggi metterà il messaggio di nuovo nella coda ed attendenderà un tempo variabile prima di tentare nuovamente.

Se la connessione va a buon fine il server PHP ci risponderà con il risultato della richiesta. Questa verrà nuovamente memorizzata nella coda dei messaggi come risposta pronta per essere inviata al client corrispondente all'ID della richiesta originale.

Nel caso della comunicazione tra il gestore di messaggi e il server avremo una comunicazione sincrona anzichè asincrona.

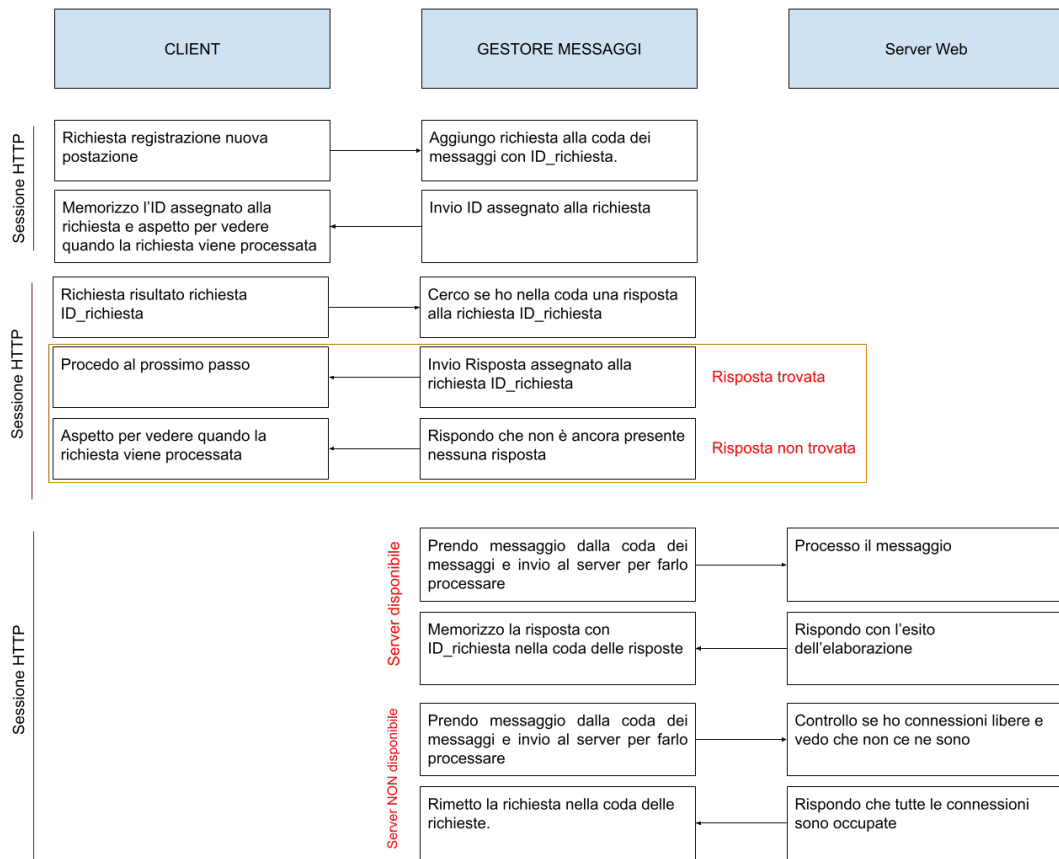


Figura 2.2: Esempio comunicazione con gestore messaggi

2.3 Diagrammi degli stati

Per comprendere meglio quali sono i messaggi che saranno necessari per la comunicazione con il server useremo dei diagrammi degli stati.

Sul client, per l'esecuzione di un test la sequenza delle operazioni che questo dovrà eseguire è la seguente:

1. Registrazione postazione sul server PHP*
2. Inserimento del codice della prova da svolgere nella pagina web da parte dell'utente
3. Invio richiesta dati del test in base al codice inserito*
 - Se il codice non è corretto scrivo Codice non corretto e lo faccio inserire di nuovo

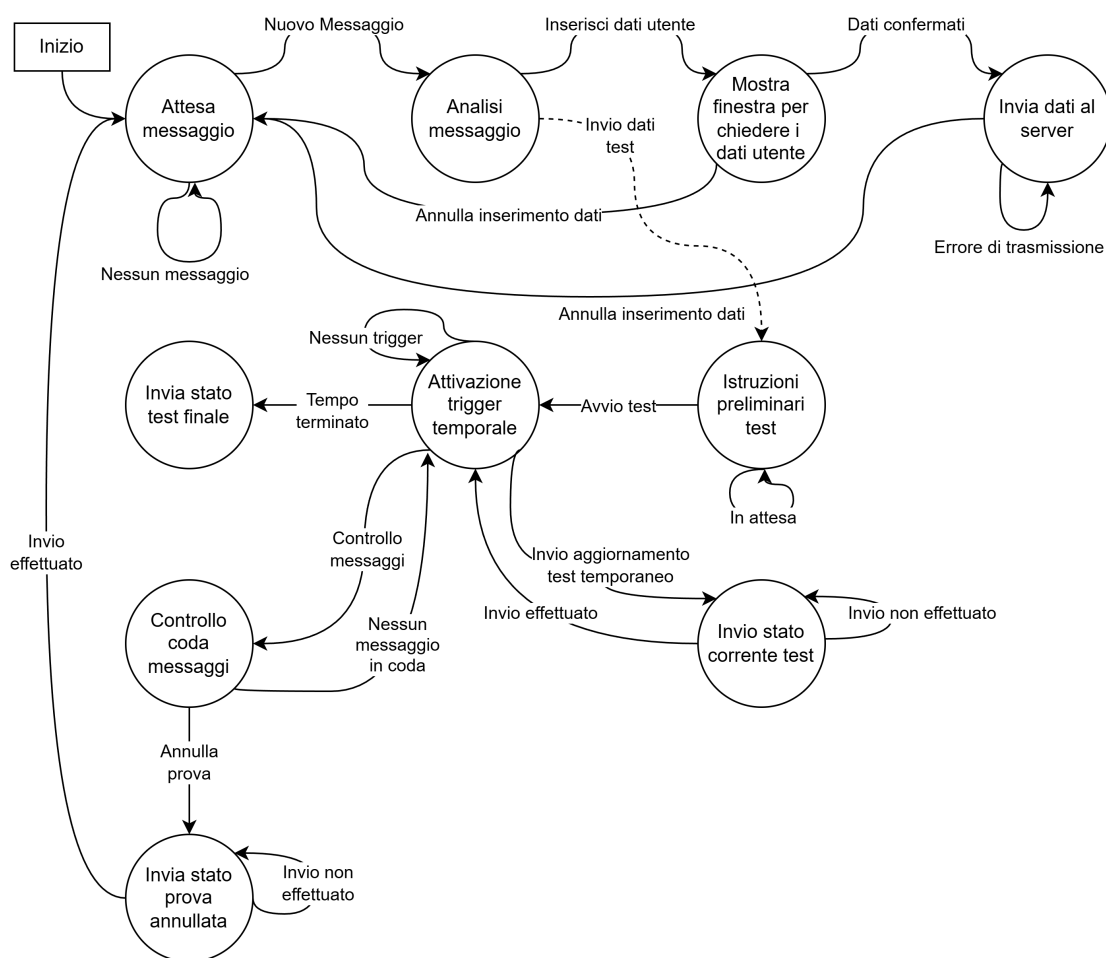


Figura 2.3: Diagramma degli stati del Client

- Se il codice è corretto usa i dati ricevuti dal server per creare dinamicamente la pagina web in base alle caratteristiche del test
4. Avvio i timer per lo svolgimento del test
 - Invio dello stato temporaneo del test (ogni 30s)*
 - Controllo se ci sono messaggi o azioni richieste dal server (interruzione test, messaggio a video, ecc.)*
 5. Consegna test definitivo*
 - Consegna manuale prima della scadenza
 - Consegna per scadenza del tempo
 6. Cancellazione registrazione della postazione*

* vuol dire che c'è una richiesta HTTP

2.4 Perché scegliere Erlang

Erlang è, come indicato anche nella pagina del progetto [3], un linguaggio usato per costruire sistemi soft real-time. massivamente scalabili con requisiti di alta disponibilità. Alcuni dei suoi utilizzi riguardano le telecomunicazioni, il settore bancario, l'e-commerce, la telefonia computerizzata e la messaggistica istantanea. Il sistema di runtime di Erlang dispone di un supporto integrato per la concorrenza, la distribuzione e la tolleranza ai guasti.

Il motivo principale per cui l'ho scelto è la sua capacità di scalare usando una quantità minima di risorse. Inoltre può lavorare in modo distribuito su più macchine e può, in caso di errori nell'esecuzione del programma, ripristinare l'esecuzione in modo automatico.

2.4.1 Erlang/OTP

OTP che sta per Open Telecom Platform è un framework di sviluppo e una collezione di librerie, componenti di design e middleware che permettono di strutturare applicazioni Erlang in modo robusto e standardizzato.

OTP fornisce dei modelli pronti all'uso per gestire i compiti più comuni come ad esempio:

- **gen_server** - Per implementare la logica client-server
- **gen_statem** - Per gestire macchine a stati finiti
- **supervisor** - Fondamentale per la tolleranza ai guasti (monitora altri processi e li riavvia se falliscono)

Un concetto fondamentale di Erlang/OTP è l'albero di supervisione (*supervision tree*). Questo è una modalità di strutturazione del modello basato sull'uso di *workers* e *supervisors*.

- I *worker* sono i processi che svolgono effettivamente il lavoro.
- I Supervisor (*supervisors*) sono processi che monitorano i *worker* e fanno li fanno ripartire se ci sono problemi.

L'albero di supervisione (*supervision tree*) è una organizzazione gerarchica del codice in supervisor e worker che ci permette di progettare sistemi che sono resistenti ai guasti (*fault-tolerant*).

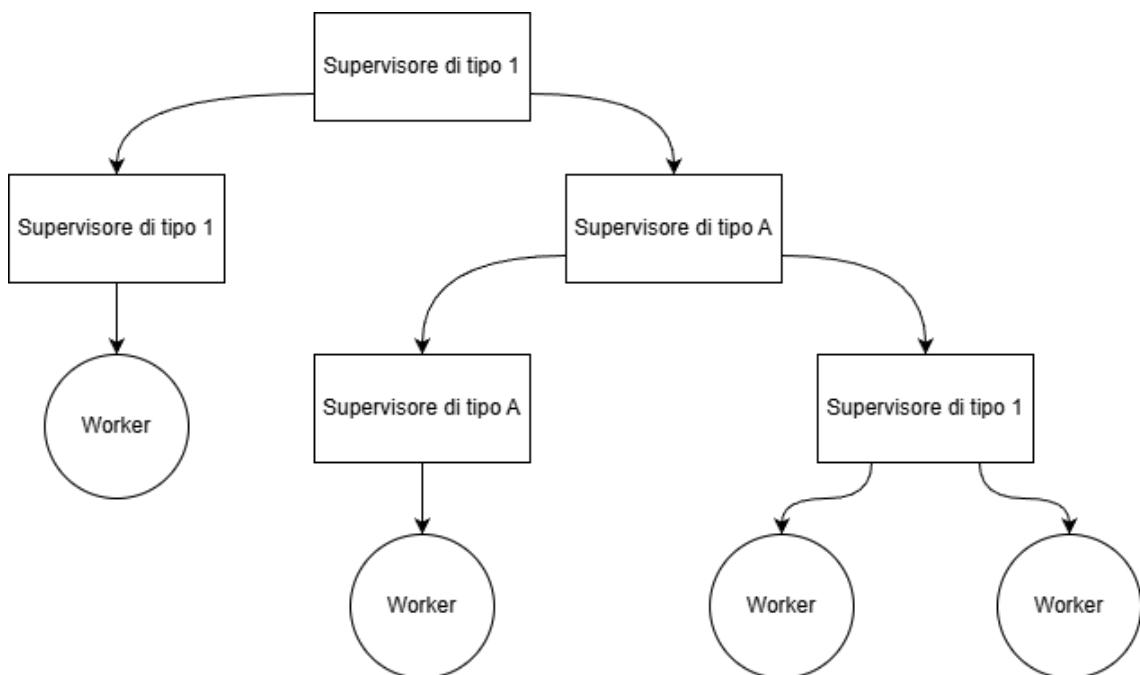


Figura 2.4: Supervision Tree

Una cosa molto interessante è che i *supervisor* e i *worker* possono essere sulla stessa macchina ma possono anche essere messi su macchine diverse. Questo ci permette di garantire che, anche se il problema riguarda fisicamente una macchina, il sistema può continuare a funzionare, creando nuovi nodi che svolgano i compiti che venivano svolti sulla macchina non più funzionante.

2.4.2 Invio e ricezione di messaggi

Altra caratteristica estremamente interessante di Erlang è la gestione nativa dei messaggi.

I vari processi (attori), infatti, possono comunicare tra di loro con un sistema di messaggistica nativo del linguaggio. Ad esempio per inviare un messaggio ad un altro processo è sufficiente il comando:

`Pid ! Messaggio`

Dove *Pid* è l'id del processo a cui inviare il messaggio.

Per ricevere messaggi, un processo utilizza il blocco `receive`, che analizza la "cassetta postale" (mailbox) del processo stesso.

```
receive
  Pattern1 -> Azioni1;
  Pattern2 -> Azioni2
end
```

a seconda del *pattern* ricevuto verrà eseguita l'azione corrispondente.

Possiamo vedere un tipi scambio di messaggio tra attori di Erlang in figura 2.5

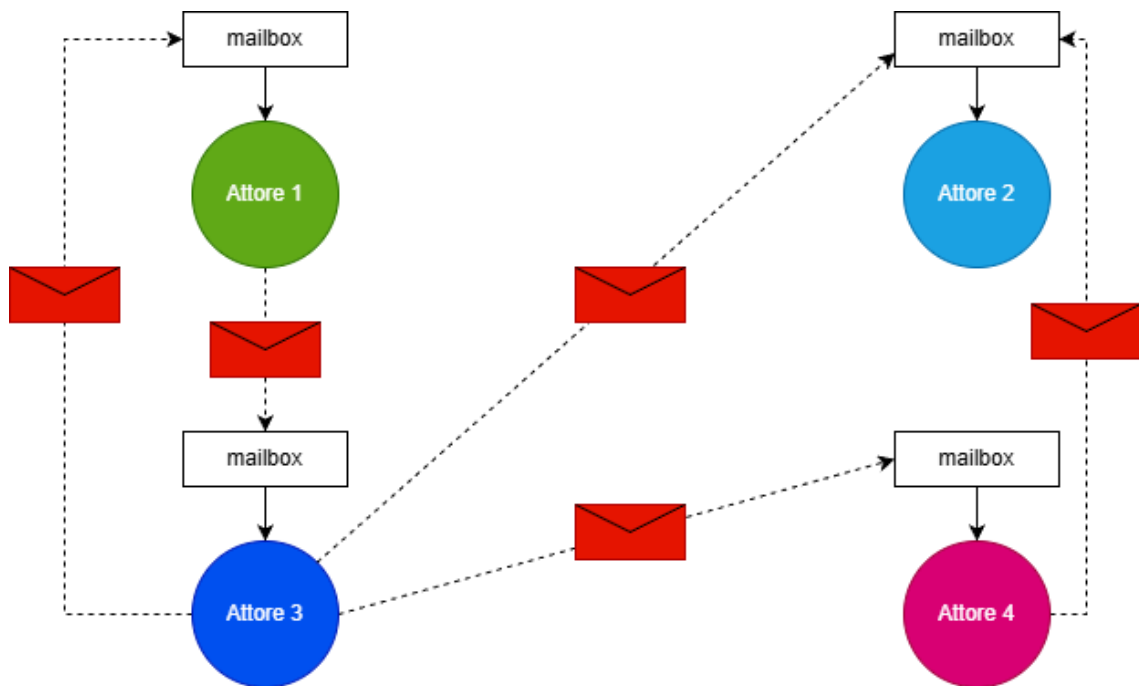


Figura 2.5: Comunicazione ad attori

In questo esempio *Attore 1* invia un messaggio ad *Attore 3* che a sua volta invia messaggi ad *Attore 1*, *Attore 2* e *Attore 4*. Infine *Attore 4* invia un messaggio ad *Attore 2*

Un esempio classico per dimostrare il funzionamento pratico della comunicazione tra attori è il ping pong, il rimpallo di messaggi tra due attori.

```
-module(ping_pong).
-export([start/0, ping/2, pong/0]).

start() ->
```

```
PongPid = spawn(fun pong/0),
spawn(fun() -> ping(3, PongPid) end).

% Processo PING - invia ping e aspetta pong
ping(0, PongPid) ->
  PongPid ! stop,
  io:format("Ping: ho finito!~n");

ping(N, PongPid) ->
  PongPid ! {ping, self()},
  receive
    pong ->
      io:format("Ping: ricevuto pong~n"),
      ping(N - 1, PongPid)
  end.

% Processo PONG - aspetta ping e risponde pong
pong() ->
  receive
    {ping, PingPid} ->
      io:format("Pong: ricevuto ping~n"),
      PingPid ! pong,
      pong();
    stop ->
      io:format("Pong: fermato~n")
  end.
```

Nella funzione `start()` vengono creati due processi che girano in parallelo. Uno per il Ping e uno per il Pong che chiameremo `PingPid` e `PongPid`. Mentre il processo `PongPid` rimane in attesa di messaggi il processo `PingPid` viene creato chiamando direttamente la funzione `ping()` a cui viene passato il Pid dell'altro processo.

La funzione `ping` di `PingPid` manda un messaggio a `PongPid` che come contenuto ha `ping, self()`. La funzione `self()` restituisce il Pid dell'attore che sta eseguendo la funzione per cui a `PongPid` arriva un messaggio del tipo `ping, Pid` che viene intercettato dalla prima clausola della `receive` nella funzione `pong()`.

`PongPid` manda a sua volta al processo che l'ha chiamato, che in questo caso è `PingPid` il messaggio `pong`.

Questa modalità, che a prima vista potrebbe sembrare complicata è estremamente semplice e, cosa più importante, è trasparente rispetto a dove si trovano i processi. Questi possono essere sulla stessa VM Erlang, su VM Erlang diverse sulla stessa macchina o su VM che si trovano su macchine che sono ubicate in paesi diversi.

2.4.3 Hot Code Swapping

Un'altra caratteristica interessante di erlang rispetto ad altri linguaggi è la sua capacità di aggiornare il codice di una applicazione mentre questa è in esecuzione (*Hot Code Swapping*), senza interrompere il servizio. I nuovi processi utilizzeranno il nuovo codice, mentre quelli vecchi continueranno con la versione precedente finchè non terminano o vengono aggiornati.

2.4.4 Casi d'uso - Whatsapp

Erlang è estremamente efficiente e ne abbiamo prova, ad esempio, dal fatto che la gestione di messaggi dell'applicazione di messaggistica *Whatsapp* viene fatta utilizzando le macchine virtuali BEAM di Erlang.

La capacità di gestire una elevata concorrenza permette ad ogni server di gestire fino a 2 milioni di connessioni contemporanee. Questo gli permette, con un team estremamente ridotto, di gestire tranquillamente oltre 100 miliardi di messaggi al giorno mantenendo, grazie alla struttura *fault-tolerant*, una enorme affidabilità e tolleranza ai guasti.

2.4.5 Esempio codice del framework OTP

Un classico esempio di implementazione del codice OTP è quello della creazione e gestione di canali di comunicazione.

Possiamo scrivere il codice in configurazione Server/Callback.

Server

```
-module(server).  
-export([start/1]).  
-export([call/2, cast/2]).  
-export([init/1]).  
  
start(Mod) ->  
spawn(server, init, [Mod]).  
  
call(Name, Req) ->  
Name ! {call, self(), Req},  
receive  
{Name, Res} ->  
Res  
end.  
  
cast(Name, Req) ->  
Name ! {cast, Req},  
ok.  
  
init(Mod) ->
```

```
register(Mod, self()),
State = Mod:init(),
loop(Mod, State).

loop(Mod, State) ->
receive
{call, From, Req} ->
{Res, State2} = Mod:handle_call(Req, State),
From ! {Mod, Res},
loop(Mod, State2);
{cast, Req} ->
State2 = Mod:handle_cast(Req, State),
loop(Mod, State2)
end.
```

Modulo di callback (ch2)

```
-module(ch2).
-export([start/0]).
-export([alloc/0, free/1]).
-export([init/0, handle_call/2, handle_cast/2]).

start() ->
server:start(ch2).

alloc() ->
server:call(ch2, alloc).

free(Ch) ->
server:cast(ch2, {free, Ch}).

init() ->
channels().

handle_call(alloc, Chs) ->
alloc(Chs). % => {Ch, Chs2}

handle_cast({free, Ch}, Chs) ->
free(Ch, Chs). % => Chs2
```

In questo esempio possiamo notare che:

- Il codice server può essere riutilizzato per costruire molti server diversi.

-
- Il nome del server, in questo esempio l'atomo **ch2**, è nascosto agli utenti delle funzioni client. Ciò significa che il nome può essere modificato senza che ciò influisca su di loro.
 - Anche il protocollo (messaggi inviati e ricevuti dal server) è nascosto. Questa è una buona pratica di programmazione e permette di modificare il protocollo senza dover modificare il codice, utilizzando le funzioni dell'interfaccia.
 - La funzionalità server può essere estesa senza dover modificare **ch2** o qualsiasi altro modulo di callback.

Capitolo 3

Strutture di comunicazione

La comunicazione tra il client, il gestore dei messaggi e il server avvengono attraverso strutture json.

3.1 Struttura JSON

La struttura JSON che viene inviata per la comunicazione tra i vari elementi dovrà avere uno schema del tipo:

```
{
  msg_type: tipo_messaggio,
  msg_content: contenuto_del_messaggio
}
```

I messaggi di ritorno avranno una struttura del tipo:

```
{
  result_type: tipo_messaggio_di_ritorno,
  result_content: contenuto_del_messaggio
}
```

Codifichiamo quindi i messaggi per il gestore dei messaggi:

Tabella 3.1: Elenco dei comandi che possono essere inviati dal client

test_message	Serve per fare un semplice test di comunicazione
register_station	La postazione vuole essere registrata sul sistema
unregister_station	La postazione vuole essere tolta dal sistema
request_code	La postazione vuole ricevere il test associato al codice inviato
send_test_content_backup	Invia al sistema una versione temporanea del test
send_test_content	Invia al sistema la versione definitiva del test

Il contenuto del messaggio dipenderà dal tipo di messaggio. Ad esempio se il tipo di messaggio è *test_message* il server si aspetterà anche il parametro *numero_iterazioni*.

Allo stesso modo dobbiamo codificare i messaggi di risposta dei server nel caso in cui l'esecuzione vada a buon fine:

Tabella 3.2: Elenco dei comandi di risposta da parte dei server

Messaggio in arrivo	Messaggio di risposta
test_message	test_result
register_station	registration_result
unregister_station	unregistration_result
request_code	test_content
send_test_content_backup	send_test_result
send_test_content	send_test_result

Se usiamo il gestore di messaggi, la comunicazione sarà asincrona per cui, una volta inviato il messaggio da parte del client, nella risposta ci sarà l'id della richiesta e, ad intervalli regolari, il client potrà verificare se la richiesta è andata a buon fine come indicato nella figura 2.2.

3.2 Struttura Record in Erlang

In erlang avremo diverse strutture interne che saranno poi convertire in JSON. Quanto utilizziamo la libreria *json* il testo json viene convertito in una mappa e viceversa. La mappa che viene resituata è del tipo:

```
Parametri_res = #{
  <<"result_type">> => <<"test_result_r">>,
  <<"result_content">> => #{
    <<"result_desc">> => <<"reading_result">>,
    <<"content">> => record_to_map(Result)
  }
}
```

In questo caso particolare il contenuto, nel campo *content*, è creato dai dati letti dal database *mnesia*. Questo restituisce un dato di tipo record che viene a sua volta convertito in mappa.

```
#{
  <<"id">>      => element(2, R),
  <<"msg_type">> => element(3, R),
  <<"payload">>  => element(4, R),
  <<"stato">>    => atom_to_binary(element(5, R), utf8),
  <<"timestamp">> => list_to_binary(io_lib:format("~p", [element(6, R)]))
}.
```

Capitolo 4

Implementazione della comunicazione

Per l'implementazione del gestore di messaggi ci affideremo al web server implementato in Erlang [3], Yaws [5].

Yaws usa un sistema simile a quello di PHP. Permette infatti di includere codice erlang direttamente all'interno della pagina WEB inserendolo all'interno delle coppia di tag `<erl>` e `</erl>`.

La comunicazione tra client e server avviene tramite il passaggio di messaggi attraverso il gestore dei messaggi in erlang.

Questa si compone di due fasi distinte: la comunicazione tra il client e il gestore di messaggi e la comunicazione tra il gestore di messaggi e il server vero e proprio.

Nella prima il client comunicherà con il gestore dei messaggi attraverso semplici connessioni http o https in get o post che verranno processate dall'interprete erlang in Yaws.

Nella seconda fase il gestore di messaggi, comunicherà con web server PHP. Per evitare che si ripresenti lo stesso problema che avevamo con troppi client implementeremo una procedura che permetta solo un numero massimo di connessioni contemporanee. Sarà il gestore dei messaggi a permettere a tutti i client di collegarsi contemporaneamente e ad inviare i messaggi ricevuti al server al ritmo che il server accetta.

4.1 Comunicazione tra client e gestore dei messaggi

4.1.1 Lato client Javascript

Visto che il client gira su un browser la comunicazioni verrà gestita tramite codice Javascript. La comunicazione avverrà usando REST.

Utilizziamo per questo come base per la comunicazione alcune funzioni javascript:

```
let cache = {};  
const request = (url, params = {}, method = "GET", customHeaders = {}) => {  
  let cacheKey = JSON.stringify({ url, params, method });  
  if (cache[cacheKey]) {  
    return cache[cacheKey];  
  }  
}
```

```
// Definiamo le opzioni base inclusi gli headers
let options = {
  method,
  headers: {
    "Content-Type": "application/json", // Default per API moderne
  }
}; if ("GET" === method) {
  url += "?" + new URLSearchParams(params).toString();
} else {
  options.body = JSON.stringify(params);
}

const result = fetch(url, options).then((response) => response.json());
cache[cacheKey] = result;

return result;
};
const get = (url, params, headers) => request(url, params, "GET", headers);
const post = (url, params, headers) => request(url, params, "POST", headers);
```

per passare i parametri al gestore dei messaggi metteremo i valori in un array

```
const dati =
{
  "msg_type": "test_messaggio",
  "contenuto": "Contenuto messaggio"
};
```

e poi li convertiranno in json

```
const json = JSON.stringify(dati);
```

infine invocheremo la funzione per inviare i dati e leggere il risultato della elaborazione

```
post("server.yaws", json, headers).then((post_res) => {
  const result_type = post_res.risposta.result_type;
  const risposta = post_res.risposta.result;
});
```

Il contenuto di risultato dipenderà logicamente dal comando che avremo inviato pur avendo la struttura di base che abbiamo definito nel capitolo 3.

4.2 Lato gestore dei messaggi

Dal lato del gestore dei messaggi dovremo gestire le richieste fatte in json ed analizzare i dati che ci verranno inviati. Nel server Yaws la funzione che viene eseguita per prima è la funzione `out()`.

```
out(Arg) ->
  Method = method(Arg),
  Risultato = try
    processa_comandi(Method, Arg)
  catch
    _ -> json:encode({
      <<"title">> => <<"Arrivato messaggio in POST">>,
      <<"method">> => <<"POST">>
    })
  end,

  %% Definiamo l'header CORS
  % Possiamo sostituire "*" con "https://tuosito.com" per maggiore sicurezza
  ACAO = {header, {"Access-Control-Allow-Origin", "*"}},

  % Torno gli header per evitare ACAO
  [
    ACAO,
    {content, "application/json", Risultato}
  ]
.
```

Alla funzione vengono passati una serie di parametri inviati dal client. Questi parametri sono passati dentro alla variabile `Arg` che è di tipo record e contiene diversi dati sulla connessione e sulla richiesta inviata al server web. Tra i più importanti troviamo:

Dati della Richiesta

- **Arg#arg.clidata**: Contiene il corpo (body) della richiesta (es. dati POST o JSON).
- **Arg#arg.req**: Contiene un altro record con i dettagli della riga di richiesta HTTP (metodo, percorso, versione).
 - **Arg#arg.req#http_request.method**: L'atomo del metodo (es. 'GET', 'POST').
 - **Arg#arg.req#http_request.path**: Il percorso URL (es. `abs_path, "/index.yaws"`).
- **Arg#arg.querydata**: Contiene la stringa dei parametri dopo il ? in un URL (es. in `test.yaws?id=1`, contiene `"id=1"`).

I parametri che a noi interessano principalmente sono il metodo di connessioni (GET o POST) perché i dati devono essere prelevati in modo differente, e le due variabili *clidata* per le chiamate in modalità POST e *querydata* per le chiamate in modalità GET.

Nel codice sopra possiamo notare una sezione che restituisce degli header aggiuntivi rispetto alla richiesta. Questi header si rendono necessari perché i browser, per gestire le policy di accesso cross-origin (CORS), impediscono collegamenti javascript che siano diretti al di fuori del proprio collegamento.

Nella nostra macchina di test, per poter eseguire contemporaneamente i server http PHP e Yaws abbiamo dovuto differenziare le porte per la connessione. Il server http PHP è stato impostato per lavorare sulla porta 80, mentre il server http Yaws è stato impostato per lavorare sulla porta 8080. Avendo messo il client sulla porta 80, per il browser qualsiasi collegamento, compresi quelli sulla stessa macchina ma su una porta diversa, vengono visti come collegamenti esterni.

Per ovviare a questo problema è necessario che il server conceda, tramite un parametro dell'header la possibilità di accesso da un sito esterno. Per far questo aggiungiamo alla risposta "Access-Control-Allow-Origin" impostando come sito autorizzato a collegarsi "*". In questo modo tutti i siti esterni sono autorizzati ad accedere al server Yaws.

4.2.1 Gestione delle diverse tipologie di connessione

La funzione *processa_comandi* si occupa di differenziare la gestione dei messaggi in base alla tipologia di connessione. In questo momento ci concentreremo sui comandi in POST anche se implementeremo comunque anche la gestione di quelli in GET per poter fare dei test di connessione in futuro.

Innanzitutto creeremo una mappa con i parametri da restituire indietro. I parametri principali saranno: *title*, *method*, *result*, risposta.

I primi due servono principalmente per verificare che la connessione funzioni, in *result* metteremo gli stessi parametri inviati dal client, anche questi per verificare il corretto funzionamento, ed infine in *risposta* ci sarà la struttura json con all'interno le informazioni che sono indirizzate al client.

Cercheremo di intercettare eventuali problemi con la gestione dei parametri tramite l'utilizzo delle clausole *try ... catch*.

La struttura dei dati verrà infine convertita in json per essere inviata indietro dal server tramite la funzione *json.encode()*. Utilizziamo la libreria *json* invece di quella proposta nel libro su Yaws [2] (*rfc4627*) perché quest'ultima necessita di installazione ed è obsoleta. Fortunatamente, a partire dalla versione **Erlang/OTP 27** è stata integrata direttamente nel linguaggio la libreria *json* che essendo nativa è anche più veloce e ottimizzata.

```
processa_comandi('POST', Arg) ->
  Parametri = #{
    <<"title">> => <<"Arrivato messaggio in POST">>,
    <<"method">> => <<"POST">>,
    <<"result">> =>
      try
```

```

        get_data(Arg)
    catch
        _:_ -> <<"result_error">>
    end,
    <<"risposta">> =>
        try
            process_data(Arg)
        catch
            _:_ -> <<"process_data_error">>
        end
    },
    Result = json:encode(Parametri),
    Result;

processa_comandi('GET', _Arg) ->
    %equivalente con le mappe di quello fatto fino ad ora
    Parametri = #{
        <<"title">> => <<"Arrivato messaggio in GET">>,
        <<"method">> => <<"GET">>,
        <<"result">> => <<"test_get">>
    },
    Result = json:encode(Parametri),
    Result;

processa_comandi(Method, Arg) ->
    Result = io_lib:format("Metodo non gestito: ~p", [Method]),
    Result
.

```

4.2.2 Memorizzazione dei messaggi

Per memorizzare i messaggi in transito tra il client e il server, avvenendo la comunicazione in modo asincrono, ci avvarremo del database nativo erlang mnesia [6] che ci permetterà di memorizzare in modo efficace i vari comandi da inviare al server PHP ed anche le risposte di questo da inviare al client javascript.

Dato che i messaggi inviati dal client e quelli resituiti dal server hanno una struttura simile e che i codici dei messaggi non sono sovrapponibili useremo una sola tabella per la coda dei messaggi.

I record della tabella avranno la struttura in tabella 4.1:

Creiamo quindi il codice che permetta di inizializzare il database che dovrà contenere la tabella mnesia.

```

out(Arg) ->
    Risultato = try

```

Tabella 4.1: Struttura dei record della tabella mnesia

Nome campo	Significato
id	Identificativo del messaggio
msg_type	Tipo di messaggio
payload	Contenuto del messaggio
stato	Stato del messaggio (non ancora usato)
timestamp	Data e ora di inserimento del dato nel DB (Serve per poter fare pulizia)

```

    esegui_inizializzazione()
catch
  _:Errore -> #{
    <<"result">> => <<"error">>,
    <<"message">> => list_to_binary(io_lib:format("~p", [Errore]))
  }
end,
[
  {header, {"Access-Control-Allow-Origin", "*"}},
  {content, "application/json", json:encode(Risultato)}
].

esegui_inizializzazione() ->
  Passi = [
    {"schema", fun crea_schema/0},
    {"mnesia", fun avvia_mnesia/0},
    {"tabelle", fun crea_tabelle/0}
  ],
  Risultati = lists:map(fun({Nome, Funzione}) ->
    Esito = try
      Funzione(),
      <<"ok">>
    catch
      _:Motivo -> list_to_binary(io_lib:format("~p", [Motivo]))
    end,
    {list_to_binary(Nome), Esito}
  end, Passi),

  application:set_env(yaws, db_inizializzato, true),

  maps:from_list([
    {<<"result">>, <<"ok">>},
    {<<"passi">>, maps:from_list(Risultati)}
  ]).

```

```

crea_schema() ->
  case mnesia:create_schema([node()]) of
    ok -> ok;
    {error, {_, {already_exists, _}}} -> already_exists
  end.

avvia_mnesia() ->
  case mnesia:start() of
    ok -> ok;
    {error, {already_started, _}} -> already_started
  end.

crea_tabelle() ->
  case mnesia:create_table(richiesta, [
    {attributes, [id, msg_type, payload, stato, timestamp]},
    {type, set},
    %{disc_copies, [node()]}
    {ram_copies, [node()]}
  ]) of
    {atomic, ok} -> ok;
    {aborted, {already_exists, _}} -> already_exists
  end.

```

Nel codice erlang del server yaws mettiamo la gestione di scrittura e lettura dei vari messaggi:

```

<<"test_message_m">> ->
  % Salva la richiesta in Mnesia
  Id = inserisci_richiesta(Tipo_Messaggio, Msg_content),
  Parametri_res = #{
    <<"result_type">> => <<"test_result_m">>,
    <<"result_content">> => #{
      <<"result_desc">> => <<Prefisso/binary>>,
      <<"id_message">> => integer_to_binary(Id)
    }
  },
  Parametri_res;

<<"test_message_r">> ->
  % Leggi la richiesta in Mnesia
  Id = maps:get(<<"Msg_content">>, Parametri, <<"default_value">>),
  case leggi_richiesta(Id) of
    {ok, Result} ->
      Parametri_res = #{

```

```

    <<"result_type">> => <<"test_result_r">>,
    <<"result_content">> => #{
        <<"result_desc">> => <<"reading_result">>,
        <<"content">> => integer_to_binary(Id)
    }
};
{error, not_found} ->
Parametri_res = #{
    <<"result">> => <<"error">>,
    <<"message">> => <<"id non trovato">>
};
{error, Reason} ->
Parametri_res = #{
    <<"result">> => <<"error">>,
    <<"message">> => list_to_binary(io_lib:format("~p", [Reason]))
}
end,
Parametri_res;

```

come si può notare, per poter mantenere la struttura che abbiamo creato per fare i primi test, cambieremo la tipologia dei messaggi che il client dovrà inviare al gestore dei messaggi: *test_message_m* è il messaggio che il client dovrà inviare al gestore dei messaggi per effettuare il test, mentre, per controllare lo stato di esecuzione del messaggio useremo *test_message_r* che restituirà, in caso positivo, l'esito dell'esecuzione.

Il client, se la risposta ricevuta dovesse essere che il messaggio non è stato ancora elaborato, attiverà di nuovo il timer per controllare lo stato di esecuzione del comando dopo un certo tempo.

4.3 Invio dei messaggi da parte del gestore dei messaggi

L'invio delle richieste dal gestore dei messaggi al server PHP può avvenire in diversi modi. Quello che utilizzeremo noi in questa prova consiste nel mandare i messaggi in modo sincrono quando un client chiede se la propria richiesta è stata processata.

In alternativa la comunicazione potrebbe avvenire attraverso un agente Erlang esterno che processi in modo autonomo. Questa opzione è più complicata da impostare ma è in grado di fornire un grado di scalabilità e affidabilità maggiore.

4.3.1 Comunicazione sincrona

Nel nostro caso sfrutteremo il fatto di aver aggiunto, alla tabella del nostro database *mnesia*, lo stato di elaborazione del messaggio. I tre stati che possiamo usare sono: *in_attesa*, *in_elaborazione* e *completata*.

Il messaggio, la prima volta che viene inserito nella tabella, avrà come valore *in_attesa* che sta ad indicare che il messaggio non è stato ancora processato.

La parte di funzione che si occupa di processare questo è il seguente:

```
<<"test_message_r">> ->
% Leggi la richiesta in Mnesia
Id = maps:get(<<"id">>, Msg_content, 0),
IdBinary = if
    is_integer(Id) -> integer_to_binary(Id);
    is_binary(Id)  -> Id;
    true           -> <<"id_non_valido">>
end,
IdNum = if
    is_integer(Id) -> Id;
    is_binary(Id)  -> binary_to_integer(Id);
    true           -> 0
end,
case leggi_richiesta(IdNum) of
{ok, Result} ->
    Status = element(5, Result), % in_attesa | in_elaborazione | completata
    NewStatus = case Status of
        "in_attesa" ->
            % non e' ancora stato elaborato il comando
            % cambiamo lo stato in in_elaborazione in modo che al prossimo passo
            % venga inviato al server PHP
            Result_elab = Result,
            binary_to_atom(<<"in_elaborazione">>);
        "in_elaborazione" ->
            % mi collego al server php per eseguire il comando
            case ConnettiAlServerPHP(Result) of
                {ok, Result_elab} ->
                    binary_to_atom(<<"completata">>);
                {busy, _} ->
                    Result_elab = Result,
                    binary_to_atom(<<"in_elaborazione">>);
            end;
        "completata" ->
            Result_elab = Result,
            binary_to_atom(<<"in_elaborazione">>)
    end,
case aggiorna_stato(Id, NewStatus) of
{atomic, ok} ->
    ok;
{aborted, Reason} ->
    {error, Reason}
end,
```

```

Parametri_res = #{
  <<"result_type">> => <<"test_result_r">>,
  <<"result_content">> => #{
    <<"result_desc">> => <<"reading_result">>,
    <<"content">> => record_to_map(Result_elab)
  }
};

```

Quando il client ci chiede se ci sono risposte alla sua richiesta, nel nostro caso *test_message*, andiamo a leggere lo stato corrispondente alla richiesta con identificativo *Id*.

Se lo status era *in_attesa* lo cambiamo con *in_elaborazione*. La volta successiva che il client ci farà la richiesta di conoscere il risultato dell'elaborazione invieremo il contenuto del messaggio al server PHP che, nel caso in cui riesca ad elaborarla la segnerà come *completata* nella tabella delle richieste.

Il client, nella iterazione successiva, trovando che quel particolare Id è stato completato chiederà al gestore dei messaggi di cancellarlo.

Invio di dati da erlang al server PHP

Per spedire i dati al server useremo, come visto, la funzione *send_http_request* che, dato il json da inviare al server, ci restituirà il risultato dell'elaborazione o il fatto che il server è sovraccarico.

Per l'invio utilizzeremo le funzioni native di Erlang per la comunicazione e in particolare *httpc:request()*.

```

send_http_request(JsonBody) ->
% Ci assicuriamo che inets sia avviato
application:ensure_all_started(inets),
application:ensure_all_started(ssl),

% Content-Type deve essere application/json
ContentType = "application/json; charset=utf-8",
Url = "http://localhost/server.php",

case httpc:request(post,
{Url, [], ContentType, JsonBody},
[{timeout, 5000}],
[]) of
{ok, [{_, StatusCode, _}, _Headers, ResponseBody]} ->
  DecodedResponse = json:decode(iolist_to_binary(ResponseBody)),
  JsonBodyReturn = #{

```

```

    <<"Status">> => integer_to_binary(StatusCode),
    <<"Risposta">> => DecodedResponse
  },
  % qui va controllato se il server era troppo occupato
  {ok, JsonBodyReturn};
{error, Reason} ->
  io:format("Errore: ~p", [Reason])
end
.

```

4.3.2 Comunicazione asincrona

Una soluzione migliore è quella di avere un modulo esterno in Erlang che venga inizializzato e che si occupi, in modo autonomo, di pescare dalla tabella mnesia e di fare la chiamata al server PHP mettendo il risultato, nel caso in cui la connessione vada a buon fine, dentro il database per poter essere recuperato dal client.

Il problema di questa soluzione è che Yaws, in configurazione standard, avvia una propria VM Erlang che è diversa e isolata da quella installata sulla macchina virtuale. Per risolvere il problema dobbiamo collegare le due VM in modo che possano condividere anche le tabelle mnesia. Questo è possibile cambiando il file di configurazione di Yaws in modo che la macchina virtuale venga lanciata con il nome e con lo stesso cookie che useremo per la VM di Erlang in modo che possano condividere la stessa rete:

Ad esempio:

```

# Avvio VM 1 (app che si collega con server PHP)
erl -name app@localhost -setcookie mio_cookie_segreto

# Avvio Yaws con lo stesso cookie
yaws --erlargs "-name yaws@localhost -setcookie mio_cookie_segreto"

```

ora che sono nella stessa rete le due VM vanno connesse:

Dalla VM della app, ci connettiamo a Yaws e verifichiamo che siano connessi

```

1> net_kernel:connect_node('yaws@localhost').

2> nodes().
% -> ['yaws@localhost']

```

e infine va collegato il database mnesia

```

% Dal nodo ESISTENTE (quello con il DB)
mnesia:change_config(extra_db_nodes, ['nuovo@localhost']).

```

Il problema di questa soluzione è che, per funzionare, Yaws deve essere configurato in modo che possa comunicare con altri worker di erlang. Questo può essere fatto in diversi modi:

1. Script di avvio (modo più comune)

```
# /etc/yaws/yaws.start oppure in uno script
yaws --erlarg "-name yaws@localhost -setcookie mio\_cookie"
```

2. File yaws.conf - parametro erl_flags

```
conferl_flags = -name yaws@localhost -setcookie mio_cookie
```

3. File /etc/yaws/yaws.conf - sezione globale

```
# Sezione globale in cima al file
logdir = /var/log/yaws
ebin_dir = /usr/lib/yaws/ebin

# Parametri erlang
erl_flags = -name yaws@localhost -setcookie mio_cookie
```

4. Variabile d'ambiente

```
export ERL_FLAGS="-name yaws@localhost -setcookie mio_cookie"
yaws --daemon
```

I due server, dato che hanno in condivisione anche il server mnesia non hanno bisogno di comunicare tra di loro con la messaggistica integrata di Erlang.

4.4 Comunicazione tra gestore dei messaggi e server PHP

4.4.1 Limitazione del numero di connessioni al server PHP

Ci sono diversi metodi per limitare il numero connessioni che il server PHP accetta dal gestore dei messaggi. Alcuni dei più comuni sono:

- Webhooks semplice (Push asincrono via HTTP)
- Semaphore (Semafori a livello di Sistema Operativo)
- Gestione tramite il Web Server (Nginx / PHP-FPM)
- Usare una tabella del database per gestire il numero di processi

Ognuno di questi ha dei vantaggi e degli svantaggi. Webhooks ad esempio ha il vantaggio che è il gestore dei messaggi ad inviare i messaggi ma rimane con lo stesso problema che abbiamo con la configurazione originale client/server; rischiamo infatti che il server di sovraccarichi. Semaphore d'altro canto richiede, per l'utilizzo dei semafori del sistema operativo, che il nostro web server si trovi su un distribuzione Linux. La gestione del numero massimo di processi modificando la configurazione del web server richiede invece la possibilità di modificare i file di configurazione, cosa che non sempre è possibile e che

dipende dalla scelta della macchina su cui far girare i nostri sorgenti PHP. Potrebbe essere una configurazione condivisa senza possibilità di cambiare i parametri di base.

L'utilizzo di una tabella del nostro database mysql o mariadb per gestire il numero massimo di richieste in arrivo dal gestore dei messaggi sembra quindi essere la soluzione migliore. Non siamo legati al sistema operativo e non abbiamo bisogno di accedere alla configurazione di PHP. Il database è già installato per memorizzare i risultati delle prove svolte.

Implementazione del codice per limitare le connessioni

Come già detto, per limitare le connessioni al server web PHP, implementeremo una gestione apposita.

Nel momento arriva una nuova connessione andremo a vedere, nella tabella delle connessioni attive, quante connessioni sono già in essere. Se il numero supera il numero massimo stabilito risponderemo alla chiamata con un messaggio che indichi il sovraccarico. In caso contrario aggiungeremo alla tabella un elemento ed eseguiremo il comando che ci è stato richiesto. Al termine del comando cancelleremo dalla tabella la riga corrispondente.

Potrebbe però capitare che, per errori nel codice PHP o altro, l'esecuzione si interrompa in modo inaspettato lasciando nella tabella una riga non più valida. L'accumulo di queste righe impedirà la ricezione di nuovi messaggi da parte del gestore dei messaggi. Implementeremo quindi una procedura di emergenza che, ad intervalli casuali di tempo, cancelli gli elementi della tabella più vecchi. Possiamo stabilire, come tempo massimo di permanenza nella tabella, un minuto. Sarà infatti molto improbabile che ci siano processi che richiedano più di una trentina di secondi per essere eseguiti.

Creiamo quindi la tabella nel database:

```
CREATE TABLE processi_attivi (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  token_univoco VARCHAR(64), -- Per sicurezza extra nella cancellazione  
  data_inizio TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  ultimo_heartbeat TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

Proseguiamo creando il codice per controllare se ci sono ancora slot disponibili:

```
function GetMaxSlots() {  
    return 10;  
}  
  
function SlotIsFree($db) {  
    // Pulizia automatica (probabilità 5% per ogni chiamata)  
    if (rand(1, 100) <= 5) {  
        // Elimina i record più vecchi di 1 minuto (processi probabilmente crashati)  
        $db->query("DELETE FROM processi_attivi  
            WHERE data_inizio < NOW() - INTERVAL 1 MINUTE");  
    }
```

```
}
$max_slots = GetMaxSlots();
$count = $db->query("SELECT COUNT(*) FROM processi_attivi")->fetchColumn();

if ($count > $max_slot) {
    return false;
}
return true;
}
```

Visto che è il codice che viene eseguito più spesso inseriamo qui il codice che, in modo casuale, cancella i record di processi attivi che si sono bloccati.

Aggiungiamo quindi il codice per prenotare e sbloccare uno slot di esecuzione:

```
function LockSlot($db) {
    $token = bin2hex(random_bytes(16)); // Generiamo un ID unico per questa esecuzione

    // 1. Inseriamo il record per tentare di occupare uno slot
    $stmt = $db->prepare("INSERT INTO processi_attivi (token_univoco) VALUES (?)");
    $stmt->execute([$token]);
    $mio_id = $db->lastInsertId();

    return $mio_id;
}

function UnLockSlot($db, $mio_id) {
    if (isset($mio_id)) {
        $db->prepare("DELETE FROM processi_attivi WHERE id = ?")->execute([$mio_id]);
    }
}
```

4.4.2 Ricezione e gestione dei messaggi nel server PHP

La gestione dei messaggi in PHP avviene in modo analogo a quello che abbiamo implementato in Erlang con le dovute differenze dovute al linguaggio di programmazione.

La funzione principale, dopo che i dati *json* sono stati estratti dal parametro POST è quella che, in base al tipo di messaggio ricevuto, esegue il codice necessario e torna indietro il risultato dell'elaborazione in modo che questo possa essere recuperato, al prossimo controllo, dal client javascript.

```
function GetRisposta($payload) {
    switch ($payload["msg_type"]) {
        case "test_message":
            $result = [
```

```
        "result_type" => "test_result",
        "result_content" => "result successful ".$payload["msg_content"]["numero_iterazioni"];
    };
    break;
    case "test_message_m":
        $result = [
            "result_type" => "test_result_r",
            "result_content" => "result successful ".$payload["msg_content"]["numero_iterazioni"];
        ];
        break;
    case "registra_postazione":
        $result = [
            "result_type" => "registra_posizione_result",
            "result_content" => "result successful ".$payload["msg_content"]["numero_iterazioni"];
        ];
        break;
    default:
        $result = [
            "result_type" => "not_found",
            "result_content" => $payload["msg_type"]
        ];
    }
    return $result;
}
```

Capitolo 5

Verifica della funzionalità

Per verificare il funzionamento delle varie parti abbiamo realizzato diversi file accessori elencati qui di seguito.

5.1 Verifica connessione

5.1.1 Verifica connessione di base

Come prima cosa verifichiamo se funziona la connessione di base.

Per far questo useremo il client javascript presente in appendice A.1 che, appena viene aperta la pagina, fa i due test in get visualizzando il risultato tornato, da una chiamata in GET dal server Yaws e dal server PHP (5.1).

Client di prova per verificare la comunicazione automatica con il gestore di messaggi

[Inizia invio dei messaggi al gestore messaggi.](#)

Test get Yaws: test_get

Test get PHP: ok

Test POST con parametri Yaws:

Test POST con parametri PHP:

Figura 5.1: Test comunicazione di base in modalità GET

Successivamente possiamo verificare anche la richiesta fatta in modalità POST cliccando sul link "*Inizia invio dei messaggi al gestore messaggi*" (5.2)

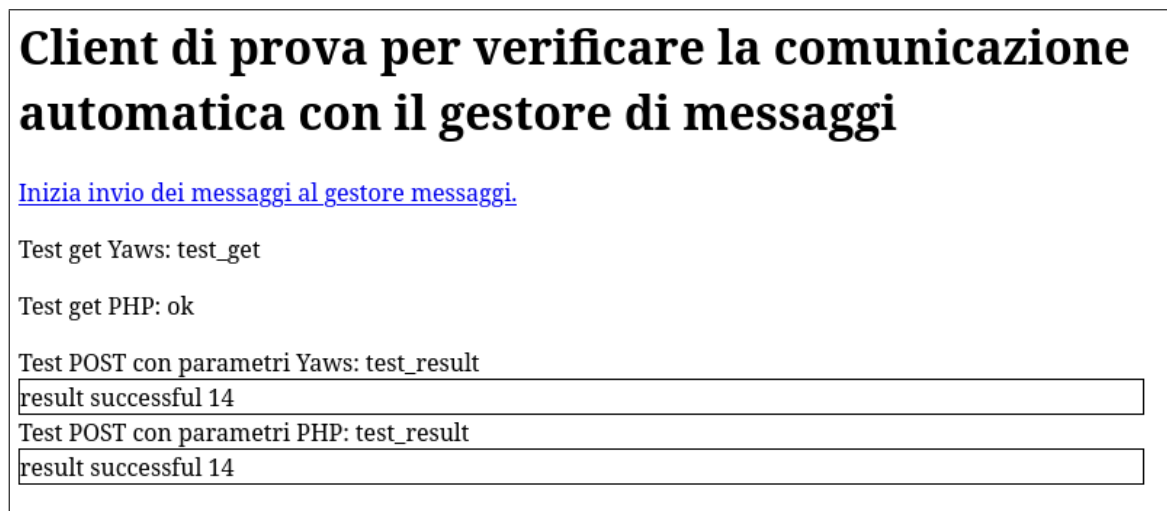


Figura 5.2: Test comunicazione di base in modalità POST

5.1.2 Verifica connessione con gestore messaggi

5.2 Stress test

5.2.1 Creazione dei file per i test

Per verificare che il nostro sistema sia affidabile creeremo un test che simuli la presenza di molti client contemporaneamente.

Ci avvarremo di tre utility presenti sulla nostra macchina Debian:

- `curl`
- `parallel`
- `jq`

Useremo ***curl*** per le comunicazioni con il server, ***parallel*** per eseguire più processi contemporaneamente in modo da simulare la connessione contemporanea da parte di più client e ***jq*** per estrarre, dalla prima richiesta, l'id del messaggio che è stato inserito nella coda.

Creiamo come prima cosa il file ***send_message_client.sh*** che invia il primo messaggio al server Yaws. Le operazioni che questo file esegue sono:

- Si collega al server con ***curl*** e invia i dati ***json*** in POST utilizzando l'ID che è stato passato al file come parametro per il numero di iterazioni.
- Mette il risultato della chiamata in una variabile per poterlo elaborare successivamente.
- Usando l'utility ***jq*** estrae, dal json contenuto nella variabile, il valore dell'id del messaggio.

- Aggiunge l'id estratto all'elenco degli id che sono stati memorizzati per poterlo usare nella seconda fase.

Il codice è il seguente:

```
#!/usr/bin/bash
ID_CHIAMATA=$1

# Supponiamo che la risposta arrivi da una chiamata curl
RISPOSTA_JSON=$(curl -X POST -d '{"msg_type":"test_message_m",
    "msg_content": {"numero_iterazioni":$ID_CHIAMATA,"test":"Sara"}}'
    http://localhost:8080/server.yaws)

# Estraiamo l'id_message usando jq
ID_ESTRATTO=$(echo "$RISPOSTA_JSON" | jq -r '.risposta.result_content.id_message')

# Assegniamo il primo argomento a una variabile con un nome chiaro
echo $ID_ESTRATTO >> elenco_id_messaggi.txt

#sleep 20
```

Passiamo quindi a creare il file *test_result_client.sh* che invia il messaggio per sapere l'esito dell'elaborazione al server Yaws. Le operazioni che questo file esegue sono:

- Mette il parametro passato sulla linea di comando nella variabile ID_RICHIESTA.
- Invia, tramite *curl* il json per conoscere lo stato della richiesta mettendo come parametro per l'ID della richiesta da controllare quello appena memorizzato.

Il codice è il seguente:

```
#!/usr/bin/bash

# Assegniamo il primo argomento a una variabile con un nome chiaro
ID_RICHIESTA=$1

curl -X POST -d '{"msg_type":"test_message_s", "msg_content":
    {"id":$ID_RICHIESTA}}' http://localhost:8080/server.yaws
```

Andiamo ora a mettere insieme le richieste e ad automatizzarle. Creiamo il file *stress_test_1.sh* che si occuperà di inviare una serie di richieste al web server e di simulare anche una prima richiesta di controllo sullo stato di elaborazione della richiesta.

Le operazioni che dovrà fare sono:

- Memorizzare in una variabile il numero di richieste da generare
- Cancellare l'elenco presente degli id di messaggi da controllare.

- Eseguire in parallelo, usando un numero di thread definito, le richieste di invio chiamando il file *send_message_client.sh* che abbiamo visto prima.
L'esecuzione di questo comando creerà, nel file *elenco_id_messaggi.txt*, un elenco degli id con cui i messaggi sono stati memorizzati nel database mnesia.
- Utilizzando il contenuto dell'elenco appena creato da passare alle singole chiamate al file *test_result_client.sh* procediamo come sopra a parallelizzare le chiamate che controllano lo stato di elaborazione del messaggio.

Il codice è il seguente:

```
#!/usr/bin/bash

# Assegniamo il primo argomento a una variabile con un nome chiaro
NUMERO_TEST=$1
# Numero di thread contemporanei
NUMERO_THREAD=50

# Azzeriamo il file con l'elenco dei messaggi da controllare
> elenco_id_messaggi.txt

seq $NUMERO_TEST | parallel -j $NUMERO_THREAD --bar --joblog send_log.log
./send_message_client.sh 2>> errori_send.txt >>output_send.txt

# ora simuliamo le interrogazioni dei client per vedere se il messaggio
# è stato processato

cat elenco_id_messaggi.txt | parallel -j $NUMERO_THREAD --bar --joblog
receive_log.log ./test_result_client.sh 2>> errori_receive.txt
>>output_receive.txt
```

Per poter fare delle valutazioni andremo a misurare il tempo di esecuzione dell'intera procedura. Creiamo quindi il file *stress_test.sh* che, usando la funzione *time*, misurerà il tempo di esecuzione della procedura memorizzandolo in un file di testo chiamato con il numero di iterazioni che stiamo eseguendo.

Il codice è il seguente:

```
#!/usr/bin/bash

# Assegniamo il primo argomento a una variabile con un nome chiaro
NUMERO_TESTS=$1

/usr/bin/time -p --output=$NUMERO_TESTS.txt ./stress_test_1.sh $NUMERO_TESTS
```

5.2.2 Esecuzione del test

Eseguiamo il test a step successivi per verificare la tenuta del server web Yaws. Partiremo da 150 richieste fino ad arrivare a 5000.

Il risultato delle prove è quello visibile in Figura 5.3.

Considerazioni

Il web server Yaws riesce a gestire tranquillamente tutte le richieste che gli sono state inviate e, come si può notare dal grafico, il tempo richiesto dal test sale in modo lineare indipendentemente dal numero di richieste che stiamo facendo.

Posto che, per essere più veritiero, il test andrebbe eseguito da una macchina esterna in modo che l'esecuzione delle richieste non utilizzi la CPU assieme al server Yaws, possiamo comunque essere soddisfatti del risultato.

Confronto con un caso reale

Volendo fare un confronto con un caso reale, la piattaforma che ho realizzato per svolgere i test del Campionato Nazionale delle Lingue[11], che è stato realizzato utilizzando la struttura classica con il server web PHP che genera l'intera pagina da mandare al client, inizia a dare problemi quando ci sono oltre 300 test in corso contemporaneamente.

Questa piattaforma, è stata studiata per essere il più leggera possibile limitando il più possibile il numero di interazioni tra i client e il server. Questi si collegano solo quando il client deve cambiare la pagina da visualizzare come ad esempio per richiedere il test o quando si cambia la pagina del test. In questo modo, durante l'esecuzione del test da parte dello studente il server non viene coinvolto quasi mai e tutto il tempo che lo studente passa su una singola pagina del test non genera traffico. Considerando delle pagine medie le interazioni saranno all'incirca ogni 3 minuti. Le probabilità che in questo lasso di tempo più client facciano richiesta di connessione al server contemporaneamente sono basse e dovrebbero abbassare il carico medio risultante sul server.

Nonostante tutte le precauzioni prese in fase di progettazione, come si diceva sopra, quando i test superano i 300 iniziano a presentarsi problemi di sovraccarico.

Nel nostro caso, nonostante la macchina di test fosse una macchina virtuale con soli 2GB di ram e 1 solo processore, ha gestito tranquillamente 5000 messaggi e, per come è strutturata, potrebbe sicuramente gestirne un numero più elevato.

#	Numero connessioni	#	Tempo
	50		5,70
	100		10,72
	150		14,48
	200		18,94
	300		32,71
	400		47,86
	500		57,08
	600		69,44
	800		90,48
	1000		102,31
	1500		184,66
	2000		216,67
	5000		556,65

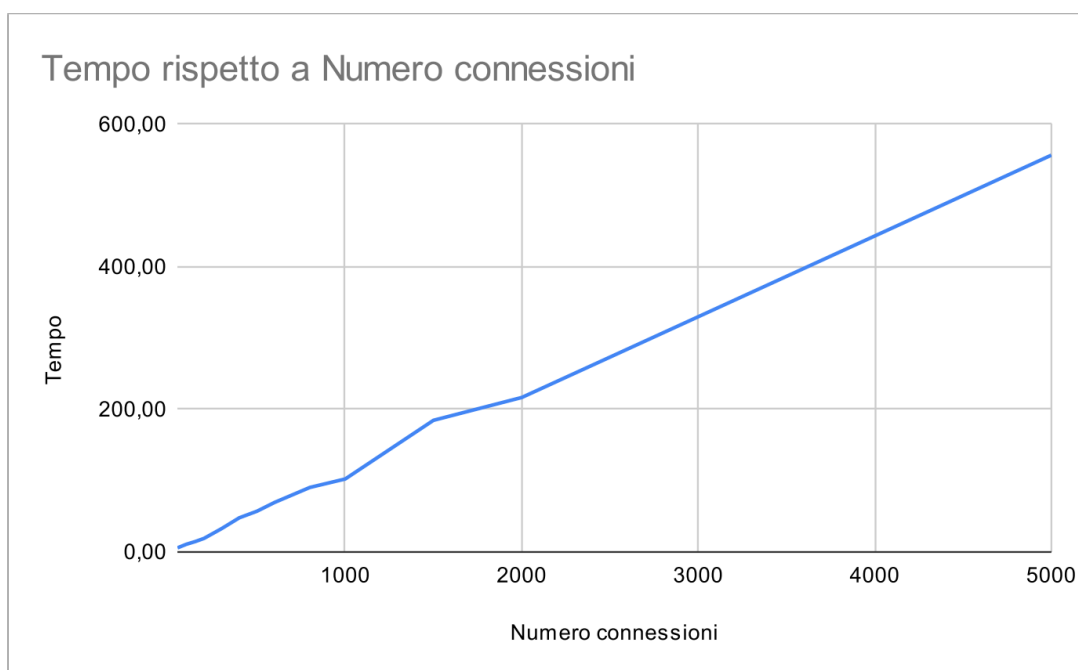


Figura 5.3: Risultati dello stress test

Capitolo 6

Conclusioni

Dopo aver effettuato le prove possiamo concludere che l'idea dell'utilizzo di un gestore di messaggi per rendere più efficiente e robusto il server PHP è realizzabile.

Durante la creazione del progetto sono emerse alcune criticità e soluzioni alternative che potrebbero essere implementate sfruttando una buona parte del codice già scritto.

Una grossa criticità di Yaws è che il codice erlang deve essere all'interno della stessa pagina ed è difficile, se non facendo operazioni particolari, usare dei moduli esterni personalizzati. Questo risulta in un codice molto lungo e poco leggibile e rende anche la manutenzione stessa dei moduli più difficoltosa.

Per quanto riguarda i miglioramenti, si potrebbe ad esempio, come citato nel capitolo 4.3.2, creare un collegamento tra la VM erlang di Yaws e altre esterne ad esso e, condividendo il database con macchine diverse da quella su cui gira il Web server Yaws, aumentarne la resilienza ed affidabilità.

Un passo in avanti, utilizzando sempre il server Yaws, potrebbe essere quella di spostare tutta la parte di elaborazione dati e scrittura e lettura dei dati nella tabella mnesia, sul codice Erlang nativo e di usare il web server solo per l'accesso da parte dei client. Potremmo in questo caso sfruttare la messaggistica interna di Erlang per inviare le richieste ricevute dai client ai vari attori (worker). In questo modo riusciamo ad alleggerire il carico sul Single Point Of Failure che nel nostro caso è il Web Server Yaws.

Un ultimo miglioramento che si potrebbe fare è di riscrivere completamente la parte di gestione delle chiamate web direttamente nel nostro codice senza utilizzare il server Yaws. In questo caso il vantaggio sarebbe che non è necessario modificare la configurazione del web server per renderlo parte della rete Erlang. Anche in questo caso, buona parte del codice che abbiamo già scritto potrebbe essere riutilizzato.

Il fatto di avere tutto in Erlang nativo ci permette anche di superare la limitazione dettata da Yaws e dividere il nostro codice in diversi moduli rendendolo più leggibile e manutenibile.

In conclusione, nonostante Erlang sia un linguaggio un po' ostico a causa della sua sintassi e del modo in cui alcune strutture della programmazione devono essere ripensate per poter essere implementate, la sua efficienza lo rende il linguaggio più adatto per progetti di questo tipo.

Appendice A

Appendice A

A.1 Client javascript per Verifica connessione di base

```
<html>
<head>
<title>Client</title>
<link rel="icon" type="image/png" href="/favicon.png">
<link rel="shortcut icon" href="/favicon.ico">
</head>
<body>
<h1>Client di prova per verificare la comunicazione automatica con il
gestore di messaggi</h1>

<p><a href="javascript:StartInvioMessaggi()">Inizia invio dei messaggi
al gestore messaggi.</a></p>
<p>Test get Yaws: <span id="get"></span></p>
<p>Test get PHP: <span id="get_php"></span></p>

<div>Test POST con parametri Yaws: <span id="post_parametri"></span></div>
<div style="border: 1px solid black;" id="return_message"></div>
<div>Test POST con parametri PHP: <span id="post_parametri_php"></span></div>
<div style="border: 1px solid black;" id="return_message_php"></div>

<script src="utils.js"> </script>
<script>
var server = "http://localhost:8080/server.yaws";
var server_php = "http://localhost/server.php";

var t_invio_messaggi;
var invio_messaggi_refresh_time_first_time = 1 * 1000; /* x secondi */
var invio_messaggi_refresh_time = 10 * 1000; /* x secondi */
```

```
var numero_iterazioni = 0;

/* Inizializza la procedura per inviare i messaggi ai server */
function StartInvioMessaggi() {
    clearTimeout(t_invio_messaggi);
    t_invio_messaggi = setTimeout('InvioMessaggi()',
        invio_messaggi_refresh_time_first_time);
}

/* Funzione che invia un messaggio ai server */
function InvioMessaggi()
{
    const dati =
    {
        "msg_type": "test_message",
        "msg_content": {
            "numero_iterazioni": numero_iterazioni,
            "test": "Sara"
        }
    };
    numero_iterazioni = numero_iterazioni + 1;

    /* invio un messaggio di tipo POST al server Yaws */
    post(server, dati).then((post_res) => {
        document.getElementById("post_parametri").innerHTML =
            post_res.risposta.result_type;
        document.getElementById("return_message").innerHTML =
            post_res.risposta.result_content;
        console.log("msg-arrived:" + post_res.risposta.result_type);
        console.log("post_res :" + post_res.risposta.result_content);
    });

    post(server_php, dati).then((post_res) => {
        document.getElementById("post_parametri_php").innerHTML =
            post_res.risposta.result_type;
        document.getElementById("return_message_php").innerHTML =
            post_res.risposta.result_content;
        console.log("msg-arrived_php:" + post_res.risposta.result_type);
        console.log("post_res_php:" + post_res.risposta.result_content);
    });

    /* faccio ripartire il timer per la prossima iterazione */
    t_invio_messaggi = setTimeout('InvioMessaggi()', invio_messaggi_refresh_time);
}
```

```
/* invio un messaggio di tipo GET al server Yaws */
get(server).then((post) => {
  document.getElementById("get").innerHTML = post.result;
});

/* invio un messaggio di tipo GET al server php */
get(server_php).then((post) => {
  document.getElementById("get_php").innerHTML = post.result;
});

</script>
</body>
</html>
```

A.2 Client javascript per Verifica comunicazioni tra client e gestore dei messaggi

Bibliografia

- [1] Francesco Cesarini e Simon Thompson, *Erlang Programming*, United States of America: O'Reilly, 2009
- [2] Zachary Kessin, *Programming Web Services with Erlang: Working with Yaws*, Great Britain: O'Reilly, 2012
- [3] *Index - Erlang/OTP* [Online] (26/11/2025). [Ultimo Accesso: 26/11/2025]. Disponibile: <https://www.erlang.org/>
- [4] *The Elixir programming language* [Online] (28/12/2025). [Ultimo Accesso: 28/12/2025]. Disponibile: <https://elixir-lang.org/>
- [5] *Yaws (web server) - Wikipedia* [Online] (28/12/2025). [Ultimo Accesso: 28/12/2025]. Disponibile: [https://en.wikipedia.org/wiki/Yaws_\(web_server\)](https://en.wikipedia.org/wiki/Yaws_(web_server))
- [6] *mnesia - A distributed key-value DBMS* [Online] (07/03/2026). [Ultimo Accesso: 07/03/2026]. Disponibile: <https://www.erlang.org/doc/apps/mnesia/mnesia.html>
- [7] *MySQL* [Online] (07/03/2026). [Ultimo Accesso: 07/03/2026]. Disponibile: <https://www.mysql.com/it/>
- [8] *MariaDB Foundation* [Online] (07/03/2026). [Ultimo Accesso: 07/03/2026]. Disponibile: <https://mariadb.org/>
- [9] *The Apache HTTP Server Project* [Online] (07/03/2026). [Ultimo Accesso: 07/03/2026]. Disponibile: <https://httpd.apache.org/>
- [10] *php.net* [Online] (07/03/2026). [Ultimo Accesso: 07/03/2026]. Disponibile: <https://www.php.net/>
- [11] *Campionato Nazionale delle Lingue* [Online] (18/03/2026). [Ultimo Accesso: 18/03/2026]. Disponibile: <https://cndl.uniurb.it/>

Ringraziamenti

Desidero ringraziare il mio relatore, il Prof. Claudio Antares Mezzina, per la guida e la costante disponibilità durante lo sviluppo di questo lavoro.

Un sentito ringraziamento va al correlatore, il Prof. Alessandro Aldini, per il prezioso supporto e le osservazioni fornite.

Estendo la mia gratitudine a tutti i docenti del corso di laurea per gli insegnamenti ricevuti e per aver contribuito alla mia formazione accademica in questi anni.

Ringrazio i miei amici e colleghi per il sostegno e la condivisione di questo percorso.

Infine, un ringraziamento speciale alla mia famiglia per il supporto incondizionato che ha reso possibile questo traguardo.